

Atividade Avaliativa: Desenvolvimento de Chatbot com RAG - Documentação Técnica

Integrantes do Grupo:

Bernardo Braga Perobeli | RM 562468

Felipe Stefani Honorato | RM 563380

Igor Paixão Sarak | RM 563726

Lucca Phelipe Masini | RM 564121

Luiz Henrique Poss | RM 562177

Disciplina: Generative AI & Advanced Nets

Data: Maio de 2026

1. Escolha do Banco de Dados Vetorial

Para atender ao **Requisito 1**, Utilizamos o banco de dados vetorial **Qdrant** em modo local persistente.

- **Desempenho e Integração:** O Qdrant oferece excelente performance para busca de similaridade via distância de cosseno.
- **Infraestrutura Ágil:** A escolha pelo modo local simplifica a infraestrutura para testes em nuvem simulada (Google Colab), garantindo simultaneamente que os dados não se percam entre reinicializações do servidor (persistência em disco).
- **Escalabilidade:** A transição para um cluster Qdrant Cloud em ambiente de produção exigiria apenas a alteração dos parâmetros de inicialização do cliente.

2. Escolha do Modelo de Linguagem

Para atender ao **Requisito 2**, a arquitetura foi integrada aos serviços da **OpenAI**.

Componente	Modelo Utilizado	Justificativa Técnica
Embeddings	text-embedding-3-small (1536 dimensões)	Padrão-ouro atual para vetorização, garantindo alta coesão semântica e eficiência de custos computacionais.
Geração (LLM)	gpt-4o-mini	Configurado com temperatura baixa (0.1) para mitigação de alucinações, forçando ancoragem estrita no contexto RAG recuperado.

3. Relação das Melhorias de Design Implementadas

Visando atender ao **Requisito 3**, o código base sofreu refatorações profundas abrangendo três frentes principais de engenharia de software:

- **Estratégias de Chunking e Metadados**

A divisão simplificada baseada em número fixo de caracteres foi substituída pelo `RecursiveCharacterTextSplitter` da biblioteca `LangChain`. Essa abordagem impede a ruptura abrupta de cláusulas e preserva a continuidade lógica dos parágrafos. Adicionalmente, cada vetor inserido no Qdrant carrega metadados estruturados (como a chave de origem), habilitando futuras buscas com filtros condicionais.

- **Injeção de Dependências e Desacoplamento**

A camada de roteamento HTTP (`FastAPI`) foi isolada das regras de negócio. Através da funcionalidade `Depends` nativa do framework, injetou-se as lógicas da `OpenAI` e do `Qdrant` como dependências das requisições. Este padrão arquitetural possibilita rotinas de testes unitários robustas e reduz consideravelmente a complexidade ciclomática das funções de roteamento.

- **Tratamento de Erros e Observabilidade**

Para aumentar a resiliência em falhas de rede típicas no consumo de LLMs (como instabilidades da API externa ou falhas de disco do banco vetorial), foram implementados blocos globais de `try/except`. Em adição ao disparo de exceções `HTTPException` devidamente tipificadas, integrou-se uma rotina de rastreamento com a biblioteca logging do Python, salvaguardando a estabilidade da aplicação e assegurando visibilidade operacional sem comprometer dados sensíveis do cliente.